Offensive IT-Tester — Project Documentation

A complete reference for the project: what it is, how every layer works, the architecture, the data and model, the tooling, and the responsible-AI controls that make it defensible.

> This is the deep-dive. For *running* it, see `run.md`. For the dataset see `data/processed/DATA_CARD.md`.

Table of contents

---

1. What it is

An **autonomous-but-bounded web-application vulnerability-testing agent**, built for the *Responsible AI & Data Ethics* course. Given an **authorized target URL**, a **LangGraph**-orchestrated agent walks seven layers — authorize → live-recon → payload-selection → governance gate → fire the payloads → confirm real exploits with detection oracles → an open-source-LLM findings report — and logs every decision to a tamper-evident ledger.

The offensive capability is the smaller part. The graded emphasis is on making it **responsible**: authorization at the front, automated safety policy in the middle, honest verification of what is actually exploitable, transparency in the report, and accountability in the audit trail.

Scope firewall. The agent can only ever touch a **loopback** host on an explicit allowlist — in practice a self-owned Flask sandbox (`127.0.0.1:5000`) or DVWA in a local Docker container (`127.0.0.1:8080`). It cannot be pointed at a third-party site. This keeps it within German law (§202a/b/c StGB), GDPR, and the EU AI Act.

2. The seven-layer architecture

```
Target URL (authorized only)
     │
     ▼
[L1] Authorization ── rejected ──► STOP (out of scope)
     │ approved
     ▼
[L2] Reconnaissance      live crawl → injection points
     │
     ▼
[L3] Selection           pick payloads from the arsenal, stratified by technique
     │
     ▼
[L4] Governance gate     automated policy: fire autonomously, flag destructive
     │ approved
     ▼
[L5] Execution           fire the payload, capture the response
     │
     ▼
[L6] Detection           oracle: did the exploit actually work? + confidence
     │
     ▼
[L7] Report              local LLM writes the findings report
     │
     ▼
Audit ledger (hash-chained) records every step throughout
```

Two of the layers are **safety gates** (the responsible-AI control points): **Layer 1 (authorization)** decides *whether* the agent may act at all, and **Layer 4 (governance)** decides *how* it acts (flagging destructive payloads, optional rate limits).

3. The agent: LangGraph orchestration over tools

This is an **agent**, not a fixed script. Three things make it one:

1. **Tools are first-class.** Each layer is a `Tool` (`authorize`, `recon`, `select_payloads`, `govern`, `execute_detect`) registered in a `ToolRegistry` with a machine-readable spec (`src/agent/tools.py`, `src/agent/layer_tools.py`). The orchestrator invokes tools *by name*.
2. **LangGraph orchestrates, and it branches.** `src/agent/graph.py` builds a `StateGraph`: each node invokes one tool; the **edges are the control flow** and they **branch on observed state —** a scope rejection or a recon failure routes straight to the end, so the agent never fires when it shouldn't. Working memory is

the graph's `RunState`.

3. **The LLM can make a bounded decision.** With `--llm`, a **triage node** hands the discovered injection points to the local model, which re-prioritises them by risk. It is bounded: it can reorder but not drop points, and the governance gate still enforces safety no matter what it says (a direct answer to OWASP LLM08, excessive agency).

Audit ledger (`src/agent/audit.py`). Every node appends a record to `audit/audit.jsonl`. Each record stores `hash = sha256(prev_hash + record)`, so the entries form a chain. `AuditLog.verify()` re-walks the chain and detects any edit. It is **tamper-evident** against casual edits (see §16 for the honest limitation).

`main.py` is the CLI driver; `webui.py` is the browser UI. Both build the same agent.

4. Layer-by-layer reference

Layer 1 — Authorization (`src/authorization/authorize.py`)

The scope firewall. Approves a URL only if its host+port is on the allowlist (`config/target_allowlist.yaml`) **and** — because `require_loopback: true` — the host is a loopback address. Everything else is rejected by default. The allowlist is stored as reviewable **data**, so an examiner can see the entire scope at a glance. This is the central lawfulness control: the tool can only attack a self-owned sandbox.

Layer 2 — Reconnaissance (`src/recon/recon.py`)

A **live crawler** (requests + BeautifulSoup). It connects to the authorized target, logs in when needed (for DVWA: `admin` / `password`, sets security to `low`), and parses every `<form>` field and URL parameter to **discover** the injection points on the *running* app. For each point it records the method, parameter, a normalised **context bucket**, the candidate attack **classes**, and any **companion form fields** (e.g. a `Submit` button — DVWA only runs its query when the whole form is submitted). It **hard-fails** if the target is unreachable — no silent fallback to a stale profile.

Layer 3 — Selection (`src/intelligence/select.py`)

For each injection point, picks payloads from the labelled arsenal (`data/cleaned/dataset_final.jsonl`). The agent **never invents payloads** — it only chooses labelled ones. Selection filters by attack class and context bucket, then **stratifies by technique**: it groups candidates by `type` and takes the best `k_per_type` (=2) of *every* technique, so no technique is skipped (the fairness fix — SQLi coverage went 3/6 → 5/6). Each selected payload carries the `oracle` that will later verify it.

Layer 4 — Governance gate (`src/governance/gate.py`)

An **automated policy — no human in the loop.** The agent fires autonomously (human review does not scale to a large, variable payload count), so the gate decides **by rule.** On the authorized, disposable sandbox it **approves every payload to fire** but **flags destructive payloads** in the audit for transparency. Two automated knobs remain for a real engagement: `allow_destructive=False` (rule-based hold of destructive payloads) and `max_per_point` (a rate limit).

Layer 5 — Execution (`src/execution/execute.py`)

The only layer that sends attack input. It fires one value at one injection point (submitting the whole form via the companion fields), and captures the response body, status, and **elapsed time** (the stopwatch the timing oracle needs). It also captures a benign **baseline** response for comparison.

Layer 6 — Detection (`src/detection/oracles.py`)

The heart of "verify exploitation" — see §5.

Layer 7 — Report (`src/reporting/report.py`)

A local open-source LLM turns the run's structured facts into a plain-prose findings report: **Summary, Scope & Authorisation, Confirmed Vulnerabilities, Recommendations**. It is instructed to talk only about the vulnerabilities and how to fix them — not the testing method — and to use no Markdown. The findings themselves come from the deterministic oracles; the model only narrates. The report carries an EU AI Act Art. 50 "AI-generated" label.

5. Detection oracles — verifying exploitation

The dataset is only an *arsenal*; it has no target responses, so **success cannot be read from it** — it must be **observed on the live target.** An **oracle** is the method that watches the response and decides *"did this attack actually work?"* There are two families:

- **Compare against a baseline** — snapshot the site's normal response, attack, and confirm only if the response changed in a telltale way (`error_signature`, `differential`).
- **Plant a signal you control** — inject something recognisable and confirm only if it comes back (`marker_reflection` = a returned marker; `timing` = a deliberate delay).

Payloads map many-to-few onto oracles: hundreds of payloads produce only a handful of *kinds of evidence*. SQL injection alone spans four oracles (differential/timing/error_signature/marker_reflection) depending on the technique.

Oracle	Confirms by
`differential`	true-vs-false condition responses diverge
`error_signature`	a DB error appears that the baseline lacked
`browser_execution`	payload reflected unescaped into HTML (reflected-XSS candidate)
`timing`	response delayed far beyond baseline (`SLEEP`)
`marker_reflection`	a backend-returned marker (not mere input echo)
`out_of_band`	a callback reaches a server you control

Honest by design. When no oracle can confirm, the tool does **not** report "safe" — the finding is left unconfirmed. A stub oracle returns `None` (unproven), never `False` (secure). And on live DVWA, blind/tautology payloads that target the *wrong* DBMS/comment style (`SLEEP--`, `pg_sleep`) correctly **did not** confirm — the tool confirms only what actually works.

6. The dataset

`data/cleaned/dataset_final.jsonl` — **2,455 rows**, one input string per row.

Half	Rows	Source
attack	455	Kaggle `WEB_APPLICATION_PAYLOADS` (repaired, deduped from 500)
benign	2,000	HTTP DATASET CSIC 2010 normal traffic (sampled, seed 42)

15-field schema includes `payload` (the only model feature), `label` / `attack_class` / `type` (the targets + technique), `context` / `context_bucket` (injection point, normalised to 14 buckets), `severity` (recomputed) + `severity_original` + `severity_reason` (audit trail), `is_destructive` / `destructive_flags` (governance signals), and `oracle` (which validator confirms this technique).

The raw Kaggle file was invalid JSON (non-breaking spaces, a missing comma, a bad escape, an empty payload, 44 duplicates) — the pipeline (`preprocess/`) repairs and de-dupes it. Severity was **recomputed** from a documented `(class, technique)` policy (283/455 labels changed), keeping the original for auditability. Full provenance: `data/processed/DATA_CARD.md`.

7. The baseline model

`models/baseline.ipynb` — **char n-gram TF-IDF + Logistic Regression**, two tasks:

Task	What it does	Macro-F1	Dummy ref
A — attack vs benign	input-side "is this an attack?" detector	0.986	0.449
B — attack_class (5-way)	routes to the detection oracle	0.991	0.072

With leakage guards (payload-text-only features; `id` / `bucket` / `severity` excluded; exact-duplicate removal; stratified hold-out; `class_weight="balanced"`; a `DummyClassifier` reference). **Honest read:** the scores are high because clean benign vs syntactically loud payloads is an easy task — treat them as an upper bound. The model is **evadable by URL-encoding** (a documented weakness), so it is kept **advisory, not authoritative**: the live loop routes by the dataset's own `type`, so a model error can never silently drop a real payload. The notebook ends with the model's own fairness and risk (NIST Map→Measure→Manage) analysis.

8. The open-source LLM (HuggingFace)

`src/agent/llm.py` — an `HFClient` that runs a local open-source model via `transformers`. Everything is local: the model is pulled from the HuggingFace Hub once and cached, then runs in-process — no API key, nothing leaves the machine.

- **Model:** `Qwen/Qwen2.5-1.5B-Instruct` (set in `config/llm.yaml`). It fits a 4 GB GPU in fp16.
- **GPU vs CPU:** the code auto-detects CUDA (`device_map="auto"`, `float16`) and otherwise runs on CPU. **CUDA torch is only available for Python 3.11/3.12, not 3.14 —** hence the project runs in a `.venv311` (Python 3.11) with a CUDA build of torch. On the RTX 3050 (4 GB) the 1.5B model streams in seconds; on CPU it takes minutes (drop to `Qwen2.5-0.5B-Instruct` if CPU-bound).
- **Uses:** the Layer-7 report (streamed token-by-token in the UI) and the optional `--llm` triage.
- **Thread-safe load:** `_ensure()` uses a lock so the model loads exactly once; the Web UI loads it at server startup so every attack streams instantly.

9. Sandbox targets

- **Flask sandbox** (`sandbox/target_app.py`) — a small, deliberately vulnerable, **loopback-only** app on `127.0.0.1:5000`. One injection point per attack class. SQLi (`/sqli`) uses a string-context query so a quote produces a real error and a tautology returns all rows; XSS (`/xss`, `/comment`) reflects unescaped. CMDi/SSRF are **simulated** (they never spawn a shell or make a request) so the sandbox can't harm the host — and so no oracle can falsely confirm them.
- **DVWA** — the standard "Damn Vulnerable Web Application" in Docker on `127.0.0.1:8080` (`docker run --rm -it -p 8080:80 vulnerables/web-dvwa`). Real MySQL + a real command shell, so it genuinely confirms error-based SQLi and reflected/stored XSS. The agent logs in and sets security to `low` automatically.

Both are on the allowlist. DVWA is more credible (third-party app, real DBMS); the Flask sandbox needs no Docker and is always available.

10. The Web UI

`webui.py` — a single-file Flask "attack console" on `127.0.0.1:7000`.

- **Enter a target URL, hit Attack.** Server-Sent Events stream the run to the browser.
- **Pipeline panel** — each layer/tool lights up one at a time (paced so it animates, not a blur), with a progress bar during firing.
- **Audit & report panel** — each confirmed exploit is written to the **audit ledger** and shown as an audit-log line (`CONFIRMED sqli/tautology — at sqli · differential · medium`), ending with the chain-verified summary. Then the Layer-7 report **streams in token-by-token** as plain prose.
- **Model loads once at server startup** (Option C): `webui.py` prints `LLM ready`, then every attack streams the report instantly. The scope firewall still applies — only allowlisted loopback targets.

11. Tests

`unit_test.ipynb` — a single, self-contained weakness-detection notebook. Each section asks the one most important question about a weakness that layer could have, and checks it live (it starts the sandbox itself). Eight questions:

1. Authorization refuses an out-of-scope site.
2. Selection stratifies across multiple techniques (no coverage collapse).
3. Destructive payloads are flagged in the audit.
4. An undemonstrable oracle returns `None` (unproven), never `False` (safe).
5. A real SQL injection is confirmed via the differential oracle.
6. A point that can't really be exploited is **not** falsely confirmed.
7. The full LangGraph agent runs end-to-end and confirms ≥1 exploit.
8. The model is evadable by URL-encoding (a real, documented weakness).

`demo_walkthrough.ipynb` is a presentation notebook that runs each layer cell-by-cell with tabular output.

12. Responsible-AI & safety controls

- **Scope firewall (L1)** — allowlist + loopback-only + reject-by-default; the tool cannot reach a third party.
- **Automated governance (L4)** — fires autonomously but flags every destructive payload; one flag (`allow_destructive=False`) switches to rule-based holds for a real engagement.

- **Honest verification (L6) —** confirms only what an oracle proves; unconfirmed ≠ safe; simulated endpoints can't false-positive.
- **Bounded LLM —** the model narrates and can re-prioritise, but never decides what is safe to fire and can't invent findings (OWASP LLM08).
- **Accountability —** the hash-chained audit ledger records every decision and every confirmed exploit; `AuditLog.verify()` detects edits.
- **Transparency —** the report is labelled AI-generated (EU AI Act Art. 50).

13. Regulatory frameworks

Framework	Relevance
EU AI Act (2024/1689)	Not prohibited (Art. 5), not high-risk (Art. 6/Annex III). Live duty: Art. 50 transparency — the report is labelled AI-generated.
GDPR (2016/679)	Art. 5(1)(c) data minimisation — training data is payload strings + benign inputs; CSIC 2010 is auto-generated, so no real PII.
StGB §202a/b/c, §303a/b	Data espionage / hacking-tools / data alteration — neutralised by the self-owned sandbox + enforced loopback scoping + destructive-flagging + documented authorization.
OWASP Top 10 (A03) + **LLM Top 10** (LLM01/LLM08)	Grounds what is tested (injection) and the agent-safety concerns (prompt injection, excessive agency → bounded agency).
ISO/IEC 42001 · **NIST AI RMF**	Structure for the governance layer, risk register, and model card (Govern / Map / Measure / Manage).

14. Project structure

```
RADE/
├── documentation.md              # this file
├── README.md                     # project overview + status
├── run.md                        # macOS/Windows setup & run guide
├── requirements.txt              # deps (+ CUDA-torch install notes)
├── main.py                       # CLI agent driver
├── webui.py                      # web attack console (Flask + SSE)
│
├── config/
│   ├── paths.py                  # canonical project paths
│   ├── target_allowlist.yaml     # the scope firewall (allowed hosts/ports)
│   ├── llm.yaml                  # LLM model + generation settings
│   └── targets/                  # dvwa.yaml (crawl config) + pyapp.yaml (Flask profile)
│
├── sandbox/target_app.py         # self-owned vulnerable Flask target
│
├── data/{raw,cleaned,processed}/ # dataset + DATA_CARD.md
├── preprocess/                   # data repair → clean → bucket → severity → merge
├── models/                       # baseline.ipynb + clf_*.pkl
│
├── src/
│   ├── agent/                    # graph.py (LangGraph), tools.py, layer_tools.py, llm.py, audit.py
│   ├── authorization/authorize.py  # L1
│   ├── recon/recon.py              # L2
│   ├── intelligence/select.py      # L3
│   ├── governance/gate.py          # L4
│   ├── execution/execute.py        # L5
│   ├── detection/oracles.py        # L6
│   └── reporting/report.py         # L7
│
├── unit_test.ipynb               # weakness-detection tests
├── demo_walkthrough.ipynb        # layer-by-layer presentation
├── audit/audit.jsonl             # generated tamper-evident ledger (git-ignored)
└── reports/                      # generated LLM reports (git-ignored)
```

15. Setup & running (quick reference)

Full macOS/Windows instructions are in `run.md`. In brief (Windows, GPU):

```
# one-time: Python 3.11 venv + CUDA torch + deps
py -3.11 -m venv .venv311
.\.venv311\Scripts\activate
pip install torch --index-url https://download.pytorch.org/whl/cu124   # CUDA torch FIRST
pip install -r requirements.txt

# start a target (pick one)
python sandbox\target_app.py                                  # Flask sandbox :5000
# docker run --rm -it -p 8080:80 vulnerables/web-dvwa         # DVWA :8080

# run it
python main.py http://127.0.0.1:5000            # CLI
python main.py http://127.0.0.1:5000 --report   # + LLM report
python webui.py                                  # web UI on :7000
```

Verify GPU: `python -c "import torch; print(torch.cuda.is_available())"` → `True`. Keep the model at **1.5B** (3B needs >4 GB VRAM).

16. Honest status & limitations

- **The baseline model is advisory** and evadable by encoding; it is deliberately never the authority on what gets fired.
- **The LLM report is narration —** small local models can phrase things imperfectly; the findings are always the deterministic oracles' output, recorded in the audit ledger.
- **Python 3.14 has no CUDA torch wheels —** GPU acceleration requires the Python 3.11/3.12 (`.venv311`) environment.

The through-line: the project prefers to **prove a few things truthfully and state its limits** rather than claim broad coverage it can't back up — which is the responsible-security judgement the course is built around.